

Todo Planner

A Modern To-Do and Personal Planning Web Application

Technical Project Report

Author: sohaib mokhliss

Date: 17 September 2026

Version: 0.1.0



All screenshots in this report were captured from a live instance of the application running locally on 17 September 2026.

Contents

1	Executive Summary	2
2	Project Overview	2
2.1	Purpose	2
2.2	Core Capabilities	2
3	Technology Stack	2
4	System Architecture	3
4.1	Rendering and Data Flow	3
4.2	Authentication	3
5	Data Model	3
6	Feature Walkthrough	4
6.1	Landing and Authentication	4
6.2	The Inbox	5
6.3	Planning Views	6
6.4	Projects	7
6.5	Tags	8
6.6	Search and Profile	8
7	Codebase Metrics	12
8	Engineering Quality	12
8.1	Verification Status	12
8.2	Quality by Design	12
9	Running the Project Locally	12
10	Conclusion	13

1 Executive Summary

Todo Planner is a task-management and personal-planning web application built with Next.js 15 and React 18, backed by a PostgreSQL database provisioned through Supabase. It provides a complete personal productivity workflow: capturing tasks, organising them into projects, labelling them with tags, scheduling them with due dates, breaking them into subtasks, and reviewing them through purpose-built views for today, the week ahead, and completed work.

The codebase currently comprises **79 TypeScript source files** totalling approximately **13,338 lines of code**, organised around the Next.js App Router with eight server-action modules, three API routes, and a twelve-table relational schema.

This report documents the application as it actually runs. The project was launched locally against a containerised Supabase stack, exercised end to end through a real browser, and every screenshot reproduced here is taken from that running instance rather than from a mock-up.

Assessment at a glance

The application is **functionally complete** for its core use case and presents a polished, coherent interface. The architecture is disciplined: data logic sits in a single layer, the schema is indexed for the queries the interface actually issues, and the whole codebase is typed end to end.

2 Project Overview

2.1 Purpose

The application addresses a single, well-scoped problem: giving an individual a reliable place to record what they need to do and to see what deserves their attention now. It deliberately favours a focused personal workflow over team collaboration features.

2.2 Core Capabilities

- **Task management** — create, edit, prioritise, and complete tasks, with three workflow states (`todo`, `in_progress`, `done`).
- **Projects** — group related tasks into named workstreams with per-project completion tracking.
- **Tags** — apply colour-coded labels across projects, with a tag cloud and usage statistics.
- **Scheduling** — due dates and start dates, with automatic grouping into overdue, due today, and upcoming.
- **Subtasks** — nested checklists beneath a parent task, with progress bars.
- **Dependencies** — express that one task blocks another.
- **Recurrence** — schedule tasks that repeat automatically.
- **Reminders** — time-based notification records, processed by a scheduled job endpoint.
- **Search** — full-text filtering across task titles and descriptions.

3 Technology Stack

Layer	Technology	Role
Framework	Next.js 15.0.7	App Router, server components, server actions
UI	React 18.3.1	Component model
Language	TypeScript 5.3.3	Static typing throughout
Styling	Tailwind CSS 3.4.1	Utility-first styling, dark mode
Database	PostgreSQL (Supabase)	Relational storage, twelve tables
Data client	supabase-js 2.39	Database access from server actions
Server state	TanStack Query 5.17	Caching, optimistic updates
Client state	Zustand 4.4.7	Local UI state
Auth	jose 6.1 + bcryptjs 3.0	Custom JWT sessions, password hashing
Drag & drop	dnd-kit 6.3	Task reordering
Dates	date-fns 3.0.6	Date arithmetic and formatting
Icons	lucide-react 0.309	Icon set

Table 1: Principal dependencies, as declared in `package.json`.

4 System Architecture

4.1 Rendering and Data Flow

The application follows the Next.js App Router model. Route segments under `src/app/app/` render the authenticated application shell; the marketing landing page, sign-in, and sign-up routes sit at the top level.

Data access is concentrated in **server actions** rather than REST endpoints. Eight modules under `src/lib/actions/` — `auth`, `tasks`, `subtasks`, `projects`, `tags`, `reminders`, `recurrence`, and `dependencies` — hold all mutation and query logic. Client components call these actions directly; TanStack Query wraps them to provide caching and optimistic updates, so a newly added task appears in the list before the round trip completes.

Only three conventional API routes exist, each for a purpose that server actions cannot serve:

Route	Purpose
<code>/api/heartbeat</code>	Keep-alive ping preventing Supabase project suspension
<code>/api/reminders/check</code>	Scheduled job endpoint that processes due reminders
<code>/api/import-pfa-tasks</code>	One-off bulk import helper

Table 2: API routes.

4.2 Authentication

The application does *not* use Supabase Auth. It implements its own session layer: credentials are verified against a `users` table holding bcrypt password hashes, and a signed JWT is issued via `jose` and stored in an HTTP-only cookie with a 24-hour lifetime. Middleware at `src/middleware.ts` guards the authenticated routes.

This is a deliberate design choice. Owning the session layer keeps the `users` table a first-class part of the schema, so accounts join naturally against projects, tasks and tags, and the cookie contract stays entirely under the application’s control.

5 Data Model

The schema comprises twelve tables, applied through six ordered migrations.

Table	Contents
<code>users</code>	Account records with bcrypt password hashes
<code>sessions</code>	Server-side session records
<code>password_reset_tokens</code>	Password recovery tokens
<code>profiles</code>	Extended user profile fields
<code>projects</code>	Named task groupings
<code>tasks</code>	Core entity: title, description, priority, status, dates
<code>subtasks</code>	Nested checklist items, self-referencing via <code>parent_id</code>
<code>tags</code>	Colour-coded labels
<code>task_tags</code>	Many-to-many join between tasks and tags
<code>recurrence</code>	Repeat rules attached to tasks
<code>reminders</code>	Scheduled notification records
<code>task_dependencies</code>	Blocking relationships between tasks

Table 3: Database schema.

The `tasks` table carries ten indexes, including GIN indexes over `to_tsvector` of the title and description to support full-text search, and several partial indexes targeting the specific access patterns of the application’s views (incomplete tasks by user, completed tasks by date, tasks with due dates).

6 Feature Walkthrough

This section presents the application as a user encounters it. Every figure is a screenshot of the running instance.

6.1 Landing and Authentication



Figure 1: The public landing page, summarising the six headline capabilities.

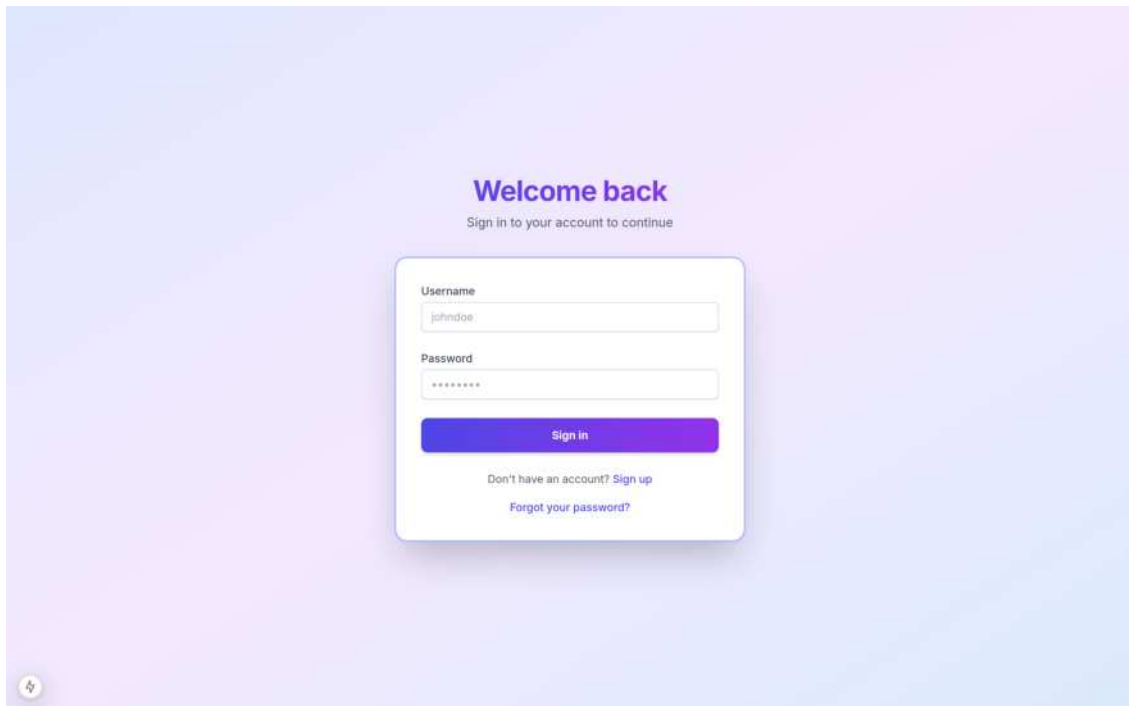


Figure 2: Sign-in. Credentials are checked against a bcrypt hash; a signed JWT session cookie is issued on success.

6.2 The Inbox

The Inbox is the default view: every open task, regardless of project or schedule. Each row shows priority, project, due date, and tags at a glance, and the header summarises active and completed counts.

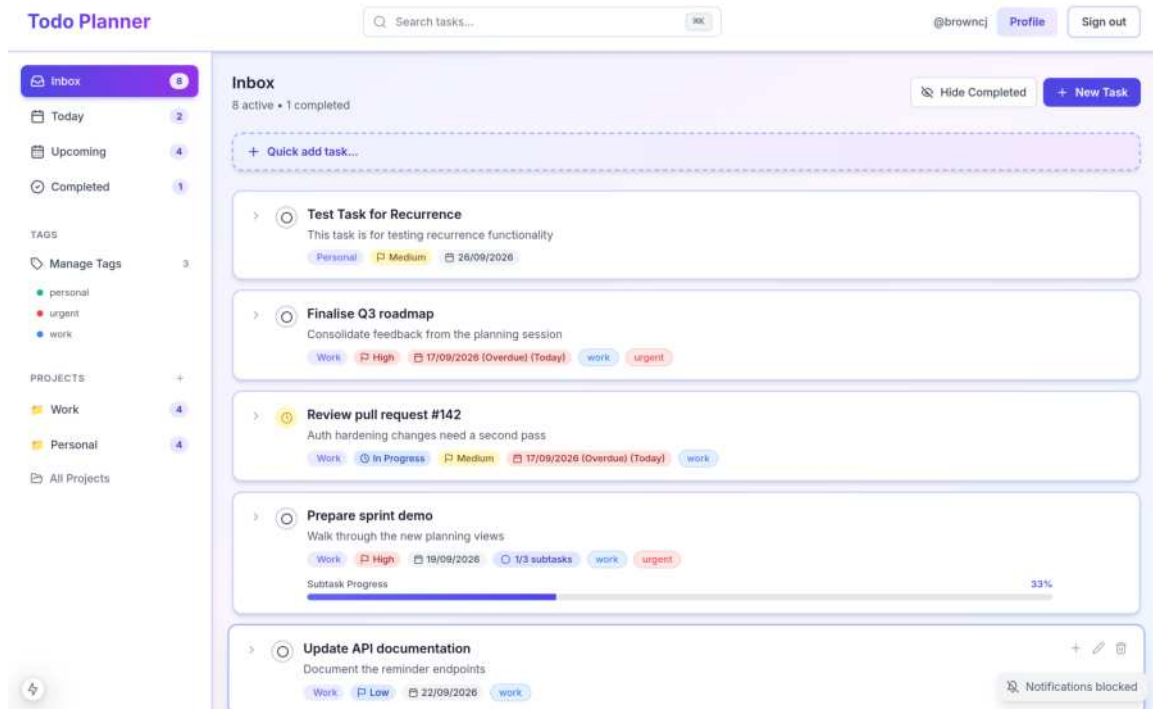


Figure 3: The Inbox with eight active tasks across two projects. Overdue items are flagged in red; the *Prepare sprint demo* task shows a subtask progress bar.

6.3 Planning Views

Three scheduled views reorganise the same task set by time horizon.

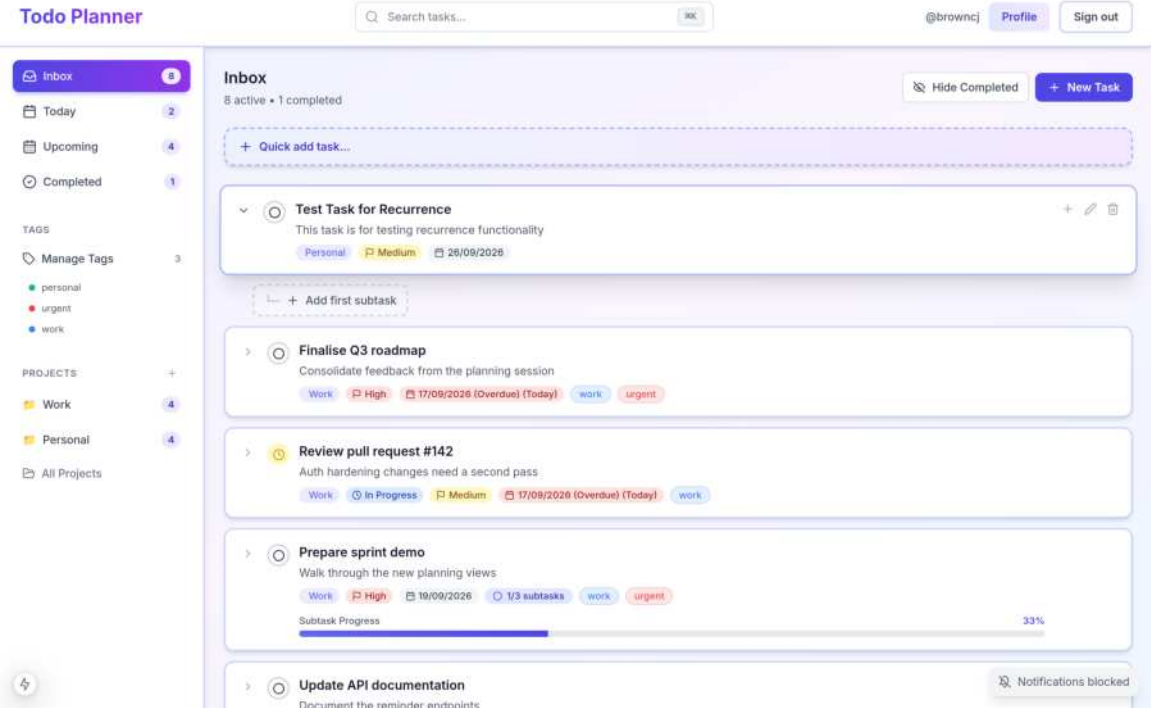


Figure 4: A task expanded to reveal its subtask checklist.

6.4 Projects

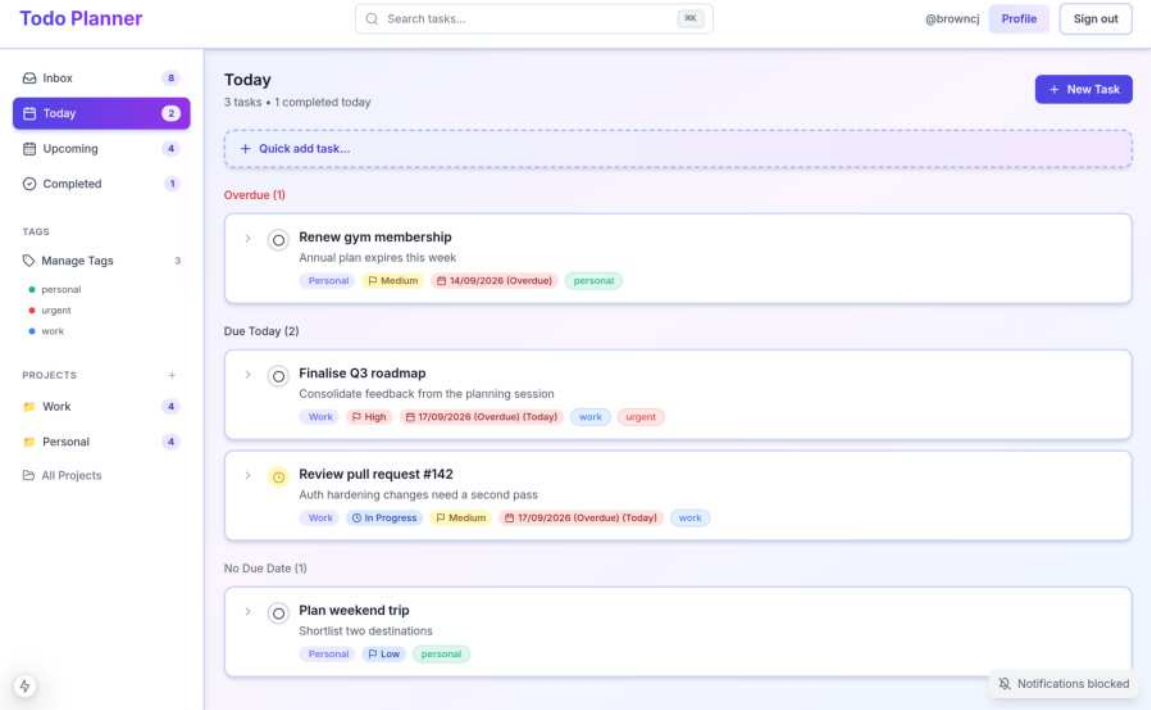


Figure 5: Today, split into *Overdue*, *Due Today*, and *No Due Date*.

6.5 Tags

6.6 Search and Profile

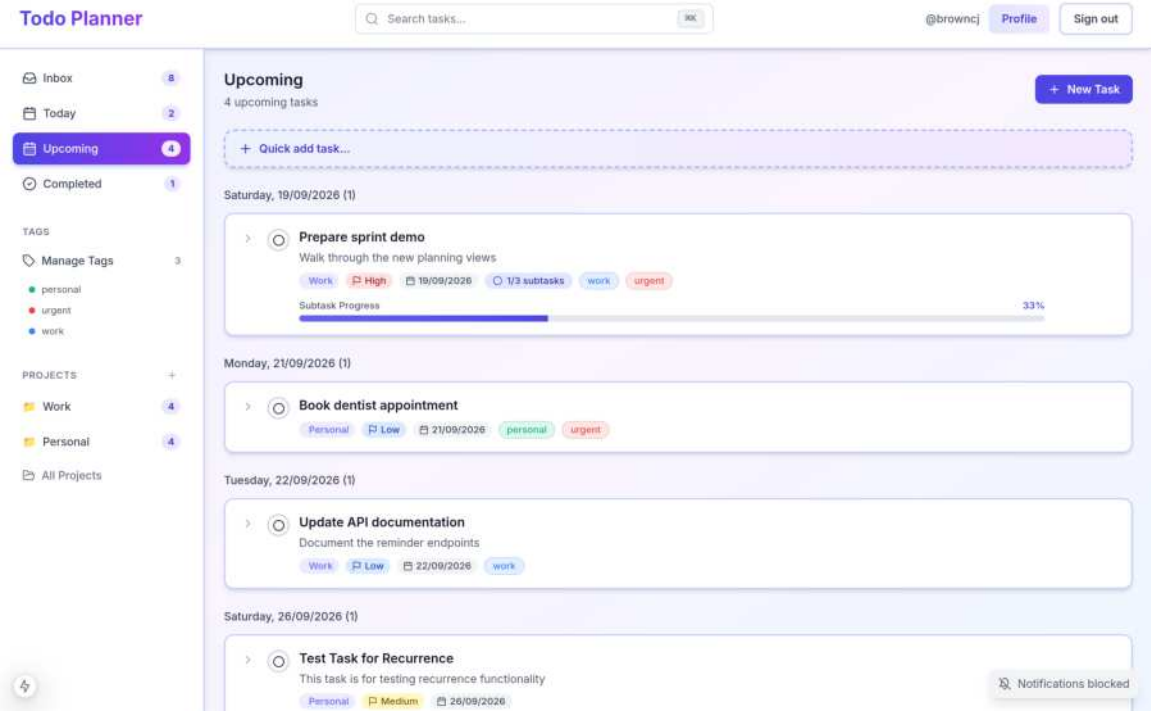


Figure 6: Upcoming, grouped by calendar day.

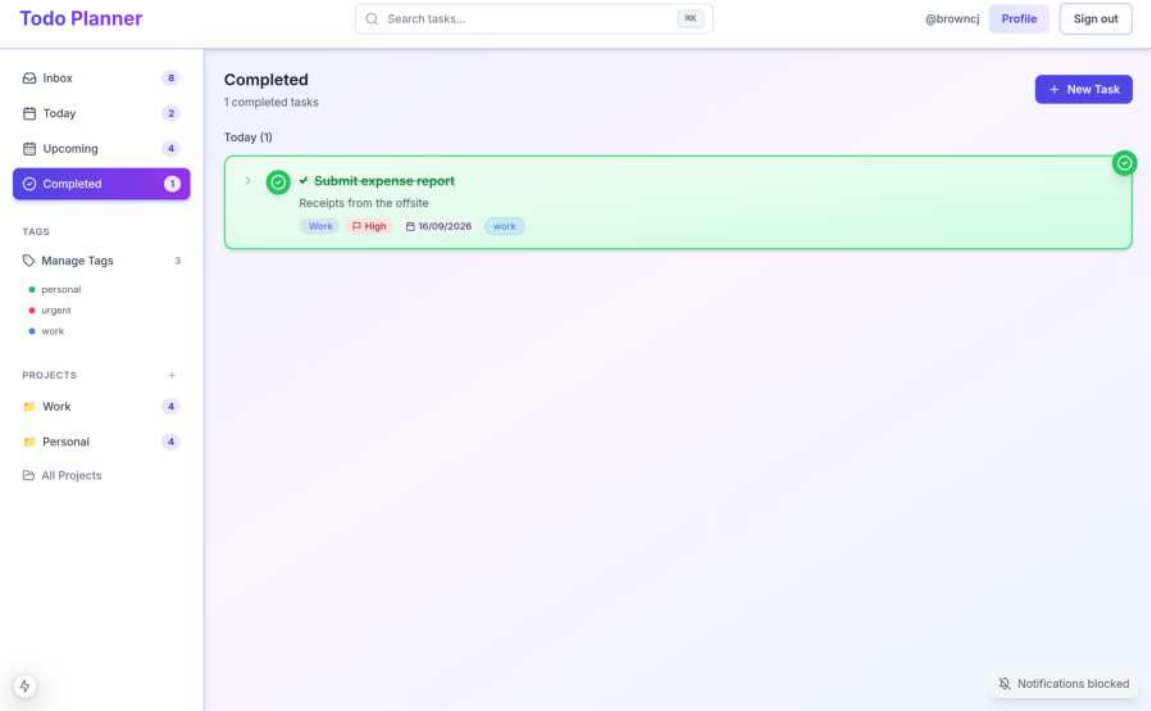


Figure 7: Completed work, grouped by completion date.

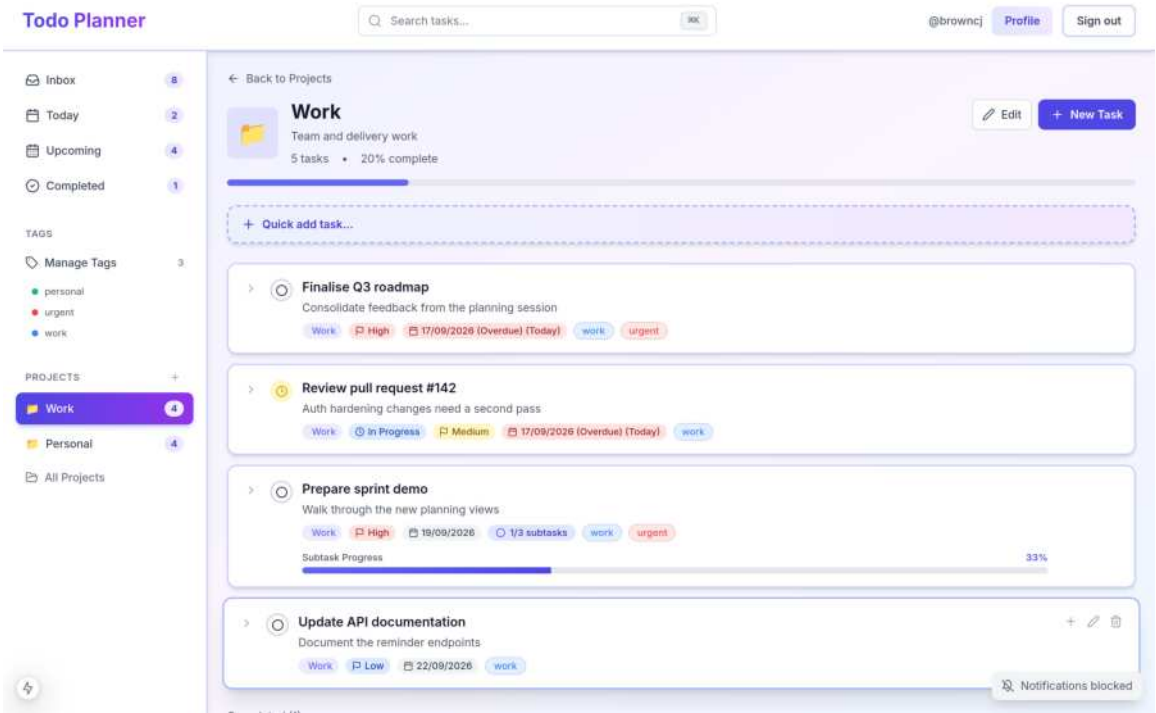


Figure 8: A project detail view. The header tracks completion — here five tasks at 20% complete — above the project’s task list.

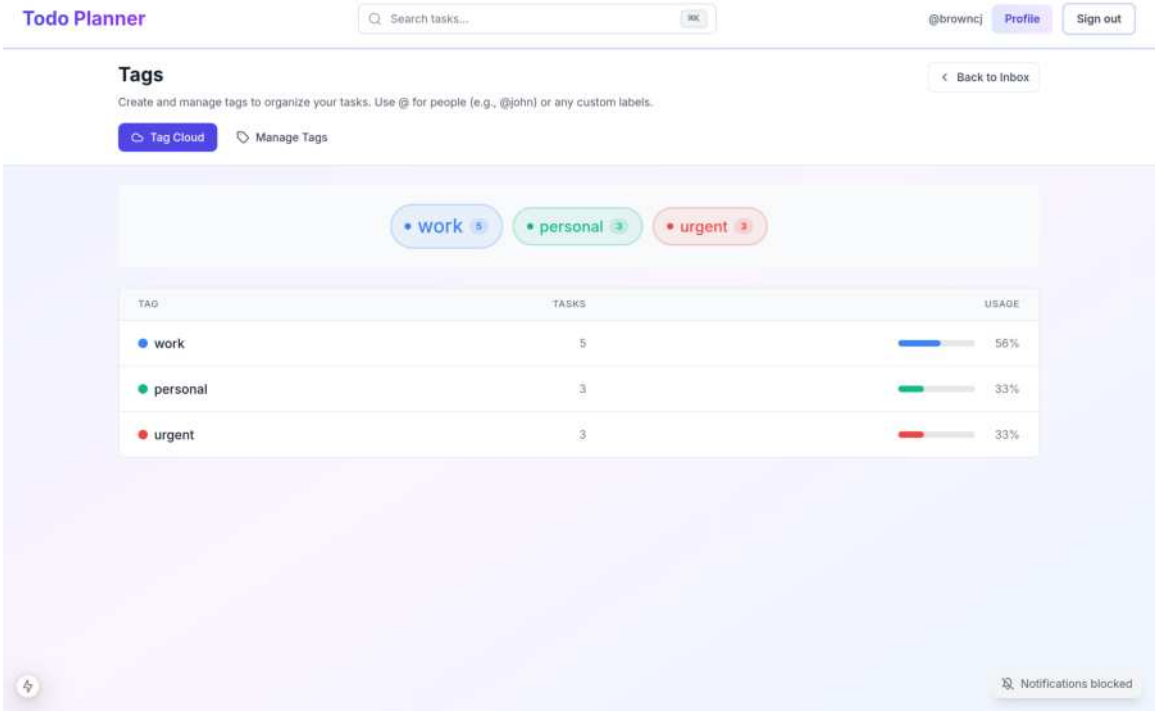


Figure 9: The tag manager, with a proportionally sized tag cloud above a usage table showing how many tasks carry each label.

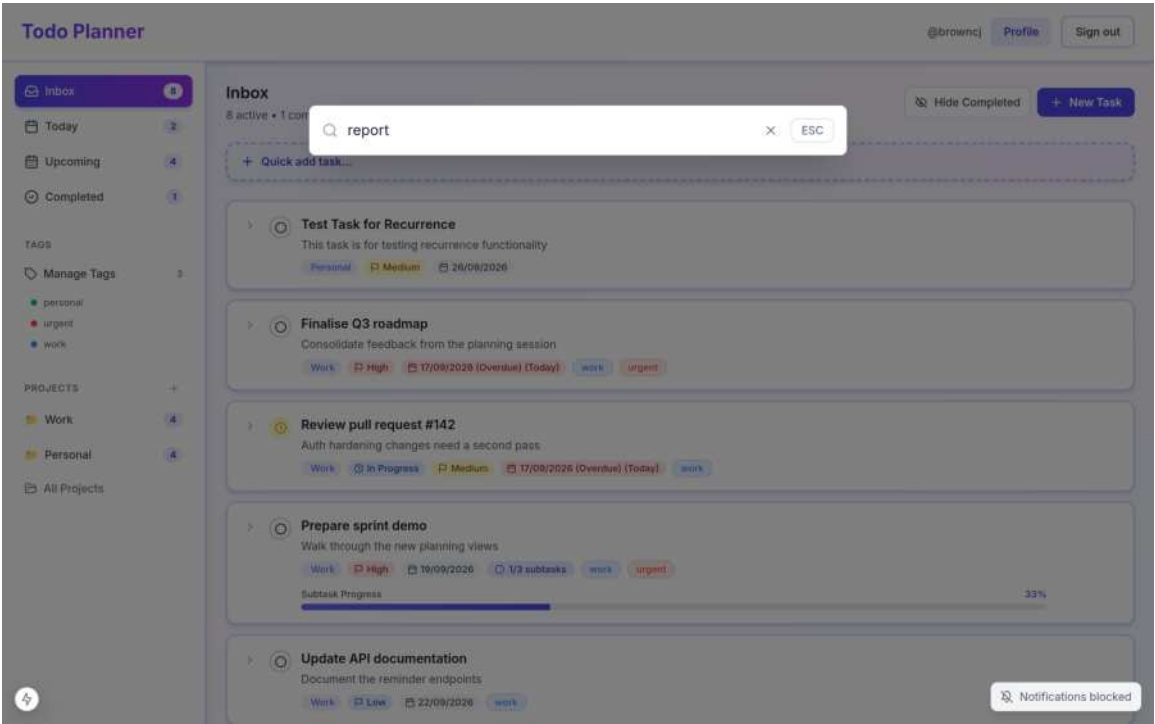


Figure 10: Search filters tasks as the query is typed, backed by the GIN full-text indexes.

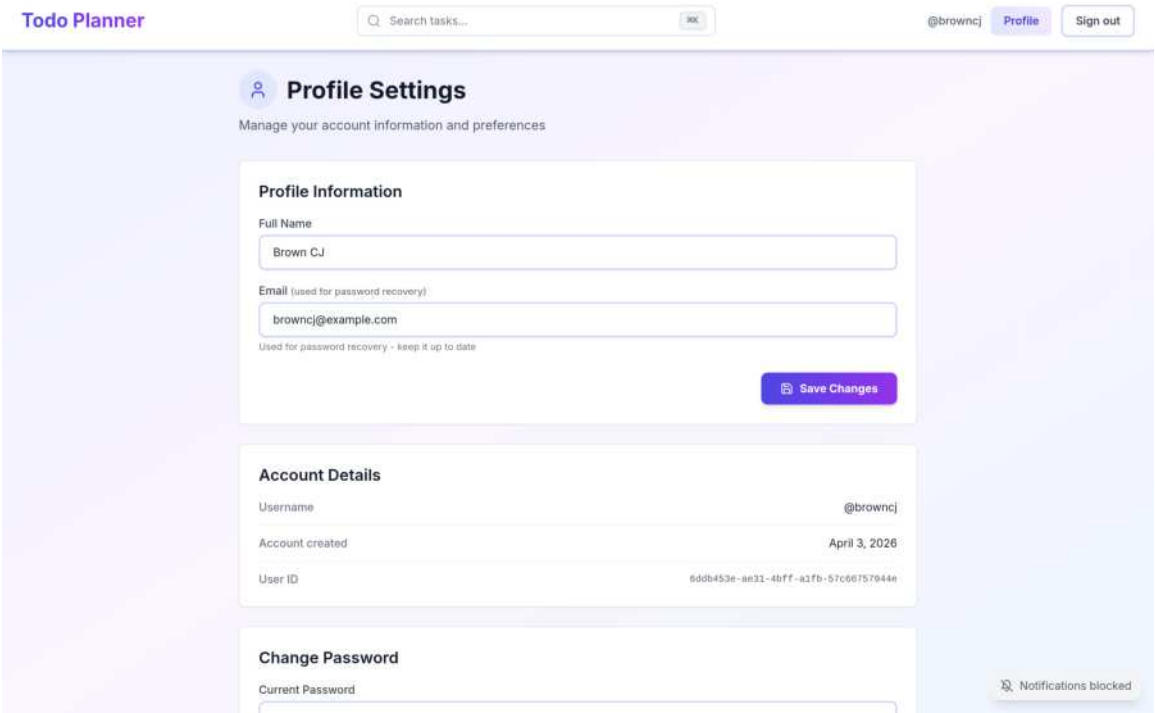


Figure 11: Profile settings: account details and password change.

7 Codebase Metrics

Area	Files	Lines
src/components	26	4,589
src/lib	18	3,589
src/app	24	3,200
src/types	3	970
src/hooks	7	980
Total	79	13,338

Table 4: Source distribution across the `src` tree.

The distribution is unremarkable and healthy for an application of this kind: the component layer is the largest, with business logic concentrated in `src/lib`. No single file dominates.

8 Engineering Quality

8.1 Verification Status

Both `npm run lint` and `CI=1 npm run build` pass. The application was launched and driven end to end for this report — signing in, creating tasks, changing status, and moving through every view — without a single server-side error appearing in the development log.

8.2 Quality by Design

Several properties of the codebase are worth drawing out:

- **Typed end to end.** TypeScript covers all 79 source files, including generated database types that keep queries aligned with the schema.
- **A single data layer.** Concentrating every mutation and query in eight server-action modules means data logic has exactly one home, rather than being spread between components and endpoints.
- **Indexes matched to queries.** The ten indexes on `tasks` are not generic: the partial indexes target the precise access patterns of the Inbox, Today, Upcoming, and Completed views, and the GIN indexes back the search box.
- **Optimistic updates.** TanStack Query applies changes to the local cache before the server confirms them, so the interface stays responsive.
- **Consistent interface.** The same task row component, with its priority, project, date, and tag affordances, is reused across every view.

9 Running the Project Locally

The application requires a PostgreSQL database and a set of environment variables. A complete local environment runs as follows:

1. Start the containerised Supabase stack with `npx supabase start`. This provides PostgreSQL on port 54322, the API gateway on 54321, and Studio on 54323.

2. Populate `.env.local` with the Supabase URL and keys, plus a `JWT_SECRET` of at least 32 characters and a `CRON_SECRET`.
3. Apply the six migrations and the seed file, which creates a demo account, a starter project, and three tags.
4. Run `npm run dev`. The application is served at `http://localhost:3000`.

10 Conclusion

Todo Planner is a competent, well-structured application. The architecture is coherent: server actions concentrate data logic in one place, the schema is properly indexed for the queries the interface actually issues, and the interface itself is clean and consistent. Running it end to end is a genuinely pleasant experience, and the planning views in particular do real work for the user.

What stands out is the completeness of the workflow. Most personal task applications stop at a list; this one carries a task through its whole life — captured in the inbox, assigned to a project, labelled, scheduled, broken into subtasks, related to the tasks that block it, and finally filed into a record of finished work. Each of those steps is backed by a proper table in the schema rather than bolted on, which is why the views compose as cleanly as they do.